# Announcements

- Programming Project 2 is assigned
- Midterm 1 is Oct 8th, the TAs will be hosting an exam review on Oct 6th from 5pm-7pm in GS 906
- The practice midterm is already released on Piazza

# Topic 4: Internal Sorting

By: Professor Hudson Lynam

# Sorting Definition

- Definition: *Sorting Algorithm*
- *A sorting algorithm, given a sequence of records* $<r_1, r_2, \ldots, r_n>$ *produces a permutation of those* $<r_1', r_2', \ldots, r_n'>$ *such that* $r_1' \leq r_2' \leq \cdots \leq r'_n$ *(ascending order)*
- Note that achieving descending order trivial
- An “internal” sort is a sort in which all records fit into RAM; that’s what we’ll be focusing on

# Why bother studying sorting?

- Java can sort for you, plenty of libraries with sorting algorithms, why bother with this?
- A few reasons:

1. Sorting is a frequently-performed task
2. Sorting algorithms provides good examples of:
   - Analysis of algorithms
   - Algorithm design principles
3. Studying several distinct approaches to one problem
4. Your language's API algorithm might not work well on your data!

# Sort Algorithm Considerations

- Best vs. Worse vs. Average cases
- Key values vs. associated data (is the algorithm stable? Does it matter for your data?)
- Key comparisons vs. data movements
- Algorithm categories: Simple ($O(n^2)$) vs. Complex ($O(n \log_2 n)$)
- Quantity of data to be sorted (internal vs. external)

# Classifying Sorting Algorithms

| Complexity → Category ↓ | Simple ($O(n^2)$) | Complex ($O(n \log_2 n)$) |
|---|---|---|
| **Exchange** | Bubble | Quick |
| **Insertion** | Insertion | Shell |
| **Selection** | Selection | Heap |
| **Merging** | --- | Two-way merge |
| **Distribution** | --- | Radix |

# Permutation Sort

- Review: In how many ways can n items be ordered?
- Answer: n!
- So, let's check all of them!

**Permutation Sort**:
While the list is not ordered:
    Permute the list

- Efficiency? O(n * n!) (it's O(n) to permute/check if sorted)
- Also known as Bogo Sort, this is far from the most efficient sorting technique…

# The Simple Sorts

- We'll cover the traditional, common three: Bubble, Insertion, and Selection sort
- But, if they're not as efficient as the complex sorts, why cover them at all?
- They're easy to implement and test
- They have little overhead when sorting relatively small lists
- Simple sorts are often used by complex sorts when sublists become small

# Bubble Sort

- The idea is that on each pass through the data, we compare overlapping pairs of elements, and exchange those not in the correct relative order.
- Bubble sort is one of the most commonly-known sorting algorithms… and also among the least efficient! It does have a cute name though.

# Bubble Sort

- Let's do two passes on this data: 33 41 7 38 16

| | | |
|---|---|---|
| *1st Pass* | [33 41] 7 38 16 | *No exchange* |
| | 33 [41 7] 38 16 | *Exchange 7 and 41* |
| | 33 7 [41 38] 16 | *Exchange 38 and 41* |
| *End 1st Pass* | 33 7 38 [41 16] | *Exchange 16 and 41* |
| *2nd Pass* | [33 7] 38 16 **41** | *Last index (41) is **set**, ex. 33/7* |
| | 7 [33 38]16 **41** | *No exchange* |
| | 7 33 [38 16] **41** | *Exchange 16 and 38* |
| *End 2nd Pass* | 7 33 16 **38 41** | *38 is **set.*** |

# Bubble Sort

- For key comparisons, what would the best, worst, and average cases be?
- For the basic algorithm, $O(n^2)$ best case. With a per-pass sortedness test, best case is O(n)
- For the worst case, let's consider how many comparisons are done per pass.
- 1st Pass does n-1 comparisons. 2nd pass does n-2 comparisons… all the way to the n-1 pass, which does 1 comparison. n-1 comparisons + n-2 comparisons + … + 1 comparisons… what would be another way of expressing this?

# Bubble Sort

- $\sum_{i=1}^{n-1} i$ key comparisons for our worst case!
- Review: $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$ , so what would our polynomial be for the # of key comparisons?
- $\sum_{i=1}^{n-1} i = \frac{n(n+1)}{2} - n = \frac{n^2+n-2n}{2} = \frac{n(n-1)}{2}$
- Which is Big-O of…?
- $O(n^2)$
- Now let's do the average case!

# Bubble Sort

- Imagine doing half of the possible n – 1 passes. Our # of key comparisons would equal the total # of comparisons minus the comparisons from the passes *not* performed
- $\sum_{i=1}^{n-1} i - \sum_{i=1}^{\frac{n-1}{2}} i = ?$
- $\sum_{i=1}^{n-1} i - \sum_{i=1}^{\frac{n-1}{2}} i = \frac{n(n-1)}{2} - ((\frac{n-1}{2})(\frac{n-1}{2} + 1))/2$
- $= \frac{(n^2-n)}{2} - \frac{n^2-1}{8} = \frac{3}{8} * n^2 - \frac{1}{2} * n + \frac{1}{8}$, which is Big-O of…?
- $O(n^2)$

# Bubble Sort

- That's key comparisons, what about data movements?
- Best Case: O(1), since nothing is moved!
- Worst Case: $O(n^2)$, if the data is in reverse order (same sum as key comparison's worst case!)
- Average Case: $O(n^2)$, again, same reasoning as for the key comparison's average case
- Finally, what about space complexity?
- Just O(n). We need to store n data values plus a few other variables
- So, when to use bubble sort? When the list is small, and you've forgotten how the other simple sorts work!

# (Cocktail) Shaker Sort: Bubble Sort Variant

- The idea is to alternate the pass directions in bubble sort. Why? Because we can move small values large distances in a single pass, not just large values.
- Quick demo: sort 5 2 3 4 1
- Pass #1 (left to right): 2 3 4 1 5
- Pass #2 (right to left): 1 2 3 4 5
- Shaker Sort is also $O(n^2)$

# Insertion Sort

- The idea is to build a sorted list one item at a time, by inserting the next item into the growing list

| | | |
|---|---|---|
| *1st Pass* | **33** [41] 7 38 16 | *Assume 33 is sorted, insert 41* |
| *End 1st Pass* | **33 41** 7 38 16 | *33 and 41 are sorted* |
| *2nd Pass* | **33 41** [7] 38 16 | *Insert 7* |
| | **33** 41 **41** 38 16 | *Remember 7, slide 41 down* |
| | 33 **33 41** 38 16 | *Slide 33 down* |
| *End 2nd Pass* | **7 33 41** 38 16 | *Place 7* |

# Insertion Sort

- Let's start with a key comparison analysis:
- Best case: O(n), if the data is already sorted, so insertions aren't necessary, just checks at the edge of the growing list
- Worst case: $O(n^2)$, if the data is in reverse order. Same summation as the worst case for bubble sort
- Average case: $O(n^2)$, same reasoning and summation as bubble sort average case

# Insertion Sort

- On to data movements analysis:
- Best case: O(1), if the data is already sorted, nothing's moved!
- Worst case: if the data is in reverse order. Each pass p has p+2 moves (save, slides(s), place).
- $\sum_{3}^{n+1} i = \sum_{1}^{n-1} i + (n+1) + (n) - 2 - 1 =$
- $\sum_{1}^{n-1} i + 2(n-1) = \frac{n(n-1)}{2} + 2(n-1)$, which is big-O of…?
- $O(n^2)$. However! If we used a linked list, the worst case is just O(n)
- Average case: $O(n^2)$.

# Insertion Sort

- Finally, space complexity analysis:
- O(n), similar to bubble sort, need store n data values, plus a few other variables. If we use a linked list? Still O(n), but a larger coefficient
- When to use insertion sort?
- When the list is small… or close to being sorted!

# Selection Sort

- The idea is to select the smallest unsorted item, and move it the front of those items.

| | | |
|---|---|---|
| *1st Pass* | 33 41 7 38 16 | *Assume 33 is smallest* |
| | 33 [41] 7 38 16 | *Scan 41, no change* |
| | 33 41 [7] 38 16 | *Scan 7, new smallest* |
| | 33 41 7 [38]16 | *Scan 38, no change* |
| | 33 41 [7]38 [16] | *Scan 16, no change* |
| *End 1st Pass* | **7** 41 33 38 16 | *Exchange 7 and 33* |
| *2nd Pass* | **7** 41 33 38 16 | *Assume 41 is smallest, scan all* |
| *End 2nd Pass* | **7 16** 33 38 41 | *Exchange 16 and 41* |

# Selection Sort

- Let's start with a key comparison analysis:
- Best case: $O(n^2)$, if the data is already sorted, we still have to check the unsorted sublist completely, every single pass. Same summation as the other two simple sort worst cases…
- We can get a Best case of O(n) with a per-pass sortedness test
- Worst case: $O(n^2)$, if the data is in reverse order. Same summation as the best case. Which means…
- Average case: $O(n^2)$

# Selection Sort

- On to a data movements analysis:
- Best case: $O(1)$, if the data is already sorted, nothing will be moved!
- Worst case: $O(n)$, if the data is in reverse order, we have to swap on every pass
- Average case: $O(n)$, if we have swap n/2 times, that's still $O(n)$
- And space complexity?
- O(n), n data values plus a few other variables
- When to use selection sort?
- When the list is small, or the data is expensive to copy

# Simple Sort Summary

| | Bubble Sort K.C. | Bubble Sort D.M | Insertion Sort K.C. | Insertion Sort D.M | Selection Sort K.C. | Selection Sort D.M. |
|---|---|---|---|---|---|---|
| **Best** | $O(n)$ | $O(1)$ | $O(n)$ | $O(1)$ | $O(n^2)$ | $O(1)$ |
| **Worst** | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ or $O(n)$ if LL | $O(n^2)$ | $O(n)$ |
| **Average** | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ or $O(n)$ if LL | $O(n^2)$ | $O(n)$ |